Light

GCSE Computer Science — Computational Thinking

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

GCSE • Edexcel • Grades 3—9 | Topic test | 30 min | 55 marks | Answered: 0 / 30

00:00 Start Pause Reset

← Back to Computational Thinking

Multiple-choice topic test for **GCSE Computer Science (Edexcel)**. You may skip questions. When you submit, you'll see your score, a grade estimate, and worked explanations. **Tip:** use the stopwatch above to time yourself.

Q1 (1 mark)

A programmer breaks down the problem 'create a game' into smaller tasks: design characters, create levels, add sound effects, implement scoring system.
What computational thinking technique is being used?

- ○ **A.** Decomposition
- ○ **B.** Encryption
- ○ **C.** Compression
- ○ **D.** Defragmentation

Q2 (1 mark)

When designing a map for a navigation app, a programmer includes roads and destinations but leaves out details like individual trees,

letterboxes, and pavement cracks.
What computational thinking technique is being used?

- ○ **A.** Data mining
- ○ **B.** Abstraction
- ○ **C.** Data compression
- ○ **D.** File encryption

Q3 (1 mark)

A programmer writes step-by-step instructions that a computer can follow to calculate the average of a list of numbers.
What has the programmer created?

- ○ **A.** A database
- ○ **B.** A spreadsheet
- ○ **C.** An algorithm
- ○ **D.** A network

Q4 (1 mark)

A programmer notices that validating email addresses, phone numbers, and postcodes all involve checking if input matches a specific format.
What computational thinking technique identifies this similarity?

- ○ **A.** Defragmentation
- ○ **B.** Decompression
- ○ **C.** Decryption

- ○ **D.** Pattern recognition

Q5 (1 mark)

To find a specific book on an unsorted shelf, you check each book one by one from left to right until you find it.
What searching algorithm does this describe?

- ○ **A.** Linear search
- ○ **B.** Binary search
- ○ **C.** Bubble sort
- ○ **D.** Merge sort

Q6 (1 mark)

To find a word in a dictionary, you open to the middle, determine if your word comes before or after, then repeat with the relevant half.
What searching algorithm does this describe?

- ○ **A.** Bubble sort
- ○ **B.** Binary search
- ○ **C.** Linear search
- ○ **D.** Insertion sort

Q7 (1 mark)

A sorting algorithm repeatedly compares adjacent pairs of items and swaps them if they are in the wrong order, continuing until no swaps are needed.

What sorting algorithm is this?

- ○ **A.** Linear search
- ○ **B.** Binary search
- ○ **C.** Bubble sort
- ○ **D.** Quick sort

Q8 (1 mark)

A sorting algorithm divides a list in half repeatedly until single elements remain, then merges the pieces back together in sorted order.
What sorting algorithm is this?

- ○ **A.** Binary search
- ○ **B.** Linear search
- ○ **C.** Bubble sort
- ○ **D.** Merge sort

Q9 (1 mark)

A table is used to track how variable values change as an algorithm executes, line by line.
What is this table called?

- ○ **A.** A trace table
- ○ **B.** A database table
- ○ **C.** A multiplication table

- ○ **D.** A periodic table

Q10 (1 mark)

In a flowchart, what shape represents a decision point where the flow can go in different directions based on a condition?

- ○ **A.** Rectangle
- ○ **B.** Diamond
- ○ **C.** Oval
- ○ **D.** Parallelogram

Q11 (1 mark)

Code-like instructions written in plain English that describe an algorithm's logic without using a specific programming language. What is this called?

- ○ **A.** Machine code
- ○ **B.** Binary code
- ○ **C.** Pseudocode
- ○ **D.** Morse code

Q12 (1 mark)

A sorting algorithm builds a sorted list one item at a time, taking each new item and inserting it into the correct position among the already-sorted items.
What sorting algorithm is this?

- ○ **A.** Merge sort
- ○ **B.** Binary search
- ○ **C.** Linear search
- ○ **D.** Insertion sort

Q13 (2 marks)

A sorted list contains 1,000 items. Using binary search, what is the maximum number of comparisons needed to find any item or confirm it is not present?

- ○ **A.** 10 comparisons — binary search halves the search space each time, so $\log_2(1000) \approx 10$
- ○ **B.** 1,000 comparisons — every item must be checked
- ○ **C.** 500 comparisons — half the items must be checked on average
- ○ **D.** 100 comparisons — one-tenth of items must be checked

Q14 (2 marks)

A student must create a program to manage a school library. They decompose it into: book database, member registration, loan tracking, overdue notifications, and search functionality.
What benefit does this decomposition provide?

- ○ **A.** It makes the program run faster
- ○ **B.** Each component can be developed and tested independently, making the overall project more manageable and enabling different team members to work on different parts simultaneously

- ○ **C.** It reduces the amount of code needed
- ○ **D.** It eliminates the need for testing

Q15 (2 marks)

A programmer creates a function called `sendEmail(recipient, subject, message)` that handles all the technical details of connecting to mail servers, authentication, and protocol formatting.
How does this demonstrate abstraction?

- ○ **A.** It makes the code more complex
- ○ **B.** It requires users to understand email protocols
- ○ **C.** It hides implementation complexity behind a simple interface — users call the function without needing to know how email protocols, servers, and authentication work internally
- ○ **D.** It prevents the function from being reused

Q16 (2 marks)

The list [5, 2, 8, 1] is sorted using bubble sort. After the first complete pass through the list, what does it look like?

- ○ **A.** [1, 2, 5, 8] — completely sorted after one pass
- ○ **B.** [5, 2, 8, 1] — unchanged
- ○ **C.** [8, 5, 2, 1] — reversed
- ○ **D.** [2, 5, 1, 8] — the largest element has bubbled to the end

Q17 (2 marks)

A programmer needs to search a list of 10,000 names that is not sorted and cannot be sorted.
Which search algorithm must be used and why?

- ○ **A.** Binary search because it is always faster
- ○ **B.** Either algorithm works equally well on unsorted data
- ○ **C.** Linear search — binary search requires sorted data, so it cannot be used on unsorted lists; linear search works on any list regardless of order
- ○ **D.** Neither algorithm can search unsorted data

Q18 (2 marks)

A flowchart contains a decision diamond asking 'Is age >= 18?' with 'Yes' leading to 'Display: You can vote' and 'No' leading to 'Display: You cannot vote yet'.
What will be displayed if age = 17?

- ○ **A.** You can vote
- ○ **B.** Both messages will be displayed
- ○ **C.** No message will be displayed
- ○ **D.** You cannot vote yet

Q19 (2 marks)

Consider this pseudocode:

```
x ← 5
y ← 3
```

```
x ← x + y
y ← x - y
x ← x - y
```

What are the final values of x and y?

- ○ **A.** x = 5, y = 3 — values unchanged
- ○ **B.** x = 8, y = 5 — simple addition and subtraction
- ○ **C.** x = 3, y = 5 — the values have been swapped
- ○ **D.** x = 0, y = 0 — values cancel out

Q20 (2 marks)

During merge sort, the list [6, 3, 8, 2] is divided. After complete division into single elements, what happens next?

- ○ **A.** Elements are randomly shuffled
- ○ **B.** Elements are merged back together in sorted order — [6] and [3] merge to [3,6], [8] and [2] merge to [2,8], then [3,6] and [2,8] merge to [2,3,6,8]
- ○ **C.** The smallest element is removed
- ○ **D.** Elements are returned in original order

Q21 (2 marks)

A programmer notices these three problems share a common pattern:
• Finding the largest number in a list
• Finding the longest word in a list
• Finding the most expensive item in a list
What general solution pattern do they share?

- ○ **A.** All follow the pattern: keep track of current 'best' value, compare each item to it, update 'best' if the new item is better
- ○ **B.** All require sorting the entire list first
- ○ **C.** All require checking only the first element
- ○ **D.** All require checking only the last element

Q22 (2 marks)

Insertion sort is applied to [4, 2, 7, 1]. After inserting the third element (7) into its correct position, what does the list look like?

- ○ **A.** [1, 2, 4, 7] — completely sorted
- ○ **B.** [2, 4, 7, 1] — the first three elements are sorted, the fourth is not yet processed
- ○ **C.** [7, 4, 2, 1] — elements are reversed
- ○ **D.** [4, 2, 7, 1] — unchanged

Q23 (2 marks)

For a list of 1,000,000 items that is already sorted, which sorting algorithm would be most efficient?

- ○ **A.** Merge sort — it always performs the same regardless of initial order
- ○ **B.** Bubble sort — it can detect sorted data and finish in one pass with no swaps
- ○ **C.** Any algorithm performs the same on sorted data

○ **D.** Sorting algorithms cannot process already-sorted data

Q24 (3 marks)

A team is designing a social media application. They identify these components:
• User authentication

```
• Profile management
• Post creation
• News feed algorithm
• Messaging system
• Notification service
```

How do decomposition and abstraction work together in this design?

○ **A.** Decomposition breaks the application into components; abstraction defines interfaces between them so each can be developed independently without knowing others' internal details — together they enable parallel development and component replacement

○ **B.** Decomposition and abstraction are the same thing

○ **C.** Only decomposition is useful; abstraction adds unnecessary complexity

○ **D.** Only abstraction is useful; decomposition fragments the project unhelpfully

Q25 (3 marks)

A database contains 10 million records that will be searched thousands of times per second. The data can be kept sorted.
Compare linear and binary search for this application.

○ **A.** Linear search is better because it is simpler to implement

- ○ **B.** Both algorithms perform identically on large datasets
- ○ **C.** Binary search is essential — linear search averaging 5 million comparisons per search is unacceptable at thousands of searches per second; binary search needs only about 24 comparisons ($\log_2$ 10 million ≈ 23.3), making high-volume searching feasible
- ○ **D.** Linear search is faster because it does not need sorted data

Q26 (3 marks)

Compare bubble sort and merge sort for sorting a large dataset of 100,000 items.

- ○ **A.** Bubble sort is faster because it is simpler
- ○ **B.** Both algorithms take exactly the same time
- ○ **C.** Sorting 100,000 items is impossible with any algorithm
- ○ **D.** Merge sort is dramatically faster — bubble sort's $O(n^2)$ means roughly 10 billion comparisons while merge sort's $O(n \log n)$ means roughly 1.7 million; merge sort uses more memory but the time difference makes it essential for large datasets

Q27 (3 marks)

A programmer's code is supposed to calculate the sum of numbers from 1 to n, but produces incorrect results. The code is:

```
total ← 0
count ← 1
WHILE count < n
    total ← total + count
    count ← count + 1
ENDWHILE
```

Using a trace table with n = 4, identify the bug.

- ○ **A.** The condition should be 'count <= n' not 'count < n' — with '<', when count reaches 4, the condition is false and the loop exits before adding 4; the trace shows total = 6 (1+2+3) instead of 10 (1+2+3+4)
- ○ **B.** The loop runs too many times
- ○ **C.** The bug is in the initial value of total
- ○ **D.** The bug is in the increment statement

Q28 (3 marks)

A programmer needs to check if a word is a palindrome (reads the same forwards and backwards, like 'radar').
Which approach demonstrates good algorithmic thinking?

- ○ **A.** Compare characters from both ends moving inward — compare first with last, second with second-to-last, continuing until middle is reached or a mismatch is found; this efficiently checks all necessary positions and stops early on non-palindromes
- ○ **B.** Check each character randomly until the answer is obvious
- ○ **C.** Only check the first and last characters
- ○ **D.** Compare the word to a list of known palindromes

Q29 (3 marks)

When should a programmer choose a flowchart versus pseudocode to represent an algorithm?

- ○ **A.** Flowcharts are always better for all purposes
- ○ **B.** Flowcharts excel at visualising control flow and decision paths (good for explaining logic visually); pseudocode better represents

complex operations, data manipulation, and translates more directly to code — choice depends on audience, algorithm complexity, and purpose

○ **C.** Pseudocode is always better for all purposes

○ **D.** The choice is arbitrary with no meaningful difference

Q30 (3 marks)

A team must develop software to manage a city's public transport network, including routes, schedules, ticketing, real-time tracking, and passenger information.
Explain how computational thinking techniques would be applied throughout this project.

○ **A.** Only decomposition is needed; other techniques are unnecessary

○ **B.** Computational thinking only applies to small problems, not large systems

○ **C.** Decomposition breaks the system into components; abstraction defines interfaces hiding complexity; pattern recognition identifies reusable solutions across similar problems (scheduling, routing); algorithmic thinking designs efficient solutions for each component — these techniques apply iteratively throughout development, not just initially

○ **D.** The techniques are applied once at the start and then forgotten

Total: **55** marks • Time: **30 min**

↑ Back to top

Submit answers

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional

Qualifications. Free to use, no login required.
About & Contact • Privacy • Terms • Accessibility • Copyright • DMCA • YouTube
WPE is free for everyone. If our resources help you or your students, you can support the project here.
© 2026 Williams Physics Education. All rights reserved.